DPL302m — Personal Task

A Technical Survey of YOLOv1: Unified, Real-Time Object Detection

Foundations and a concise
perspective on the YOLO family (v2–v12)

Author

Le Khoi Nguyen

October 2026

Abstract

You Only Look Once (YOLO) reframed object detection as a unified regression problem: a single neural network maps an entire image directly to bounding boxes and class probabilities. This report studies the original YOLOv1 paper, published at CVPR 2016, as its primary model. It explains the problem motivation, grid-based prediction, the 24-convolutional-layer architecture, responsibility assignment, training objective, and inference pipeline. The analysis connects the model's innovations to its speed, contextual reasoning, localization errors, and difficulty with clustered small objects. A shorter historical section explains how subsequent YOLO branches address representation, training, deployment, and attention, supported by a version catalogue in the appendix. The original model occupies approximately four-fifths of the main technical discussion. The implementation section and spaces for an architecture illustration and real-image inference results are reserved for later completion.

Keywords: object detection; YOLOv1; one-stage detector; convolutional neural network; real-time vision

Contents

1 Introduction **2**
1.1 Scope and organization . . . 2

2 Background and Motivation **2**
2.1 From multi-stage detection to unified prediction . . . 2
2.2 Why full-image reasoning matters . . . 2

3 YOLOv1: Unified Real-Time Object Detection **3**
3.1 Detection formulation and grid responsibility . . . 3
3.2 Output representation . . . 3
3.3 Network architecture . . . 4
3.4 Training strategy . . . 5
3.5 Inference and post-processing . . . 6

4 Key Innovations Relative to Earlier Detectors **8**

5 Experimental Results and Assessment **8**
5.1 Reported performance . . . 8
5.2 Advantages . . . 9
5.3 Limitations . . . 9

6 Typical Applications of YOLOv1 **10**

7 Simple Demonstration / Implementation Example **10**

8 Evolution Beyond YOLOv1 **10**
8.1 Reading the family as several research branches . . . 10
8.2 From box priors to multi-scale representation: v2–v3 . . . 11
8.3 Optimizing training and deployment: v4–v7 . . . 11
8.4 Revisiting the head, gradient, and post-processing: v8–v10 . . . 12
8.5 Toward attention-centric features: YOLO11–v12 . . . 12

9 Conclusion **12**

A YOLO Family Catalogue: v1–v12 **13**

1 Introduction

Object detection requires a model to answer two related questions: *what* objects are present and *where* they occur in an image. Before YOLO, many successful detectors were assembled from multiple components. A typical region-proposal pipeline generated candidate regions, extracted CNN features for each region, classified the features, refined the bounding boxes, and finally removed duplicate detections. Although accurate, this design was computationally costly and difficult to optimize as a single system [1, 2, 3].

YOLOv1 introduced an influential alternative: predict object locations and classes from the full image in one network evaluation [1]. Rather than treating detection as a collection of independent subproblems, it learns a direct mapping from image pixels to spatially separated bounding boxes and associated class probabilities. The central question of this report is how this formulation achieves real-time detection, and what constraints follow from the original design.

1.1 Scope and organization

The report focuses on the original YOLOv1 paper and its PASCAL VOC configuration. Section 2 explains the motivation; section 3 develops the main idea, architecture, and learning procedure; and sections 4 to 6 assess innovations, strengths, limitations, and applications. Section 7 is reserved for an implementation example once the required materials are available. Section 8 provides supporting historical context rather than a second full survey. Later-version references combine the supplied original papers with official software documentation for YOLOv5, YOLOv8, and YOLO11. Benchmark results in this report are attributed to the original papers; no implementation experiment is reported.

2 Background and Motivation

2.1 From multi-stage detection to unified prediction

Sliding-window detectors repeatedly evaluate a classifier at many positions and scales. Region-based systems reduce this repetition by generating proposals, but then still process a large set of candidate regions. For example, R-CNN relies on approximately 2,000 Selective Search proposals per image [2, 1]. These approaches separate proposal generation, feature extraction, classification, localization, and post-processing.

YOLOv1 removes the proposal-generation and per-region prediction stages. It processes the entire image at once and learns the following direct prediction task:

$$\text{image pixels} \longrightarrow \text{bounding boxes} + \text{confidence scores} + \text{class probabilities}. \tag{1}$$

This formulation permits end-to-end optimization of the main prediction network and makes real-time detection a central design goal.

The comparison in figure 1 describes R-CNN, not every later region-based detector. Fast R-CNN shares a convolutional feature map across proposals and is therefore more efficient than running a CNN separately on each crop [3]. YOLO's distinction remains its dense, direct full-image prediction without an external proposal stage. Likewise, "end-to-end" in the original paper concerns joint learning of the prediction network; NMS remains outside that learned mapping.

2.2 Why full-image reasoning matters

Unlike a detector that classifies isolated crops, YOLO sees the complete image during both training and inference. Full-image features provide context about object appearance, scene

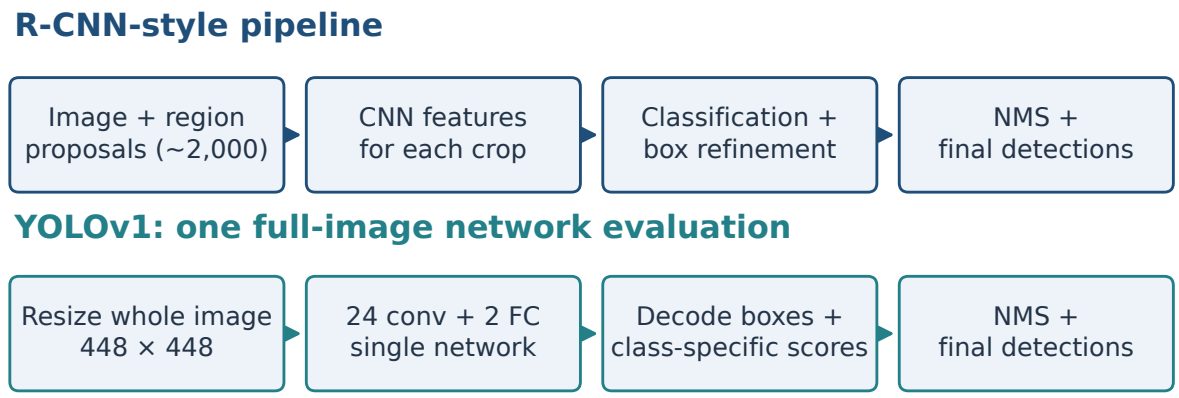


Figure 1: A proposal-based R-CNN-style workflow and YOLOv1's unified prediction workflow. Original schematic based on [2, 1]; both retain post-processing.

layout, and relationships between objects. The original paper reports that YOLO produces fewer false positives on background than Fast R-CNN, though it makes more localization errors [1].

3 YOLOv1: Unified Real-Time Object Detection

3.1 Detection formulation and grid responsibility

YOLOv1 conceptually divides the image into an $S \times S$ grid. A grid cell is responsible for an object if the object's center lies inside that cell. Importantly, the cell determines responsibility only; the predicted bounding box may extend beyond the cell boundary.

For the PASCAL VOC experiments in the original paper, the configuration is

$$S = 7, \qquad B = 2, \qquad C = 20, \tag{2}$$

where B is the number of bounding-box predictors per cell and C is the number of classes. The network therefore returns a tensor of shape

$$S \times S \times (B \cdot 5 + C) = 7 \times 7 \times 30. \tag{3}$$

3.2 Output representation

Each cell predicts $B = 2$ bounding boxes. A box has five values,

$$(x, y, w, h, \text{confidence}), \tag{4}$$

and the cell also predicts one shared vector of $C = 20$ conditional class probabilities, $P(\text{Class}_i \mid \text{Object})$. Thus, one cell produces $2 \times 5 + 20 = 30$ values.

The coordinates (x, y) describe the box center as offsets relative to the responsible grid cell. Width and height are normalized by the image dimensions:

$$w = \frac{W_{\text{box}}}{W_{\text{image}}}, \qquad h = \frac{H_{\text{box}}}{H_{\text{image}}}. \tag{5}$$

The confidence target is defined as

$$\text{confidence} = P(\text{Object}) \times \text{IoU}_{\text{pred}}^{\text{truth}}, \tag{6}$$

where IoU denotes intersection over union. At inference time, the class-specific score for box j and class i is

$$\text{Score}_{i,j} = P(\text{Class}_i \mid \text{Object})\, P(\text{Object})\, \text{IoU}_{\text{pred}}^{\text{truth}}. \tag{7}$$

The class vector is shared by the two box predictors in one cell; consequently, the two predictors are not independent class classifiers.

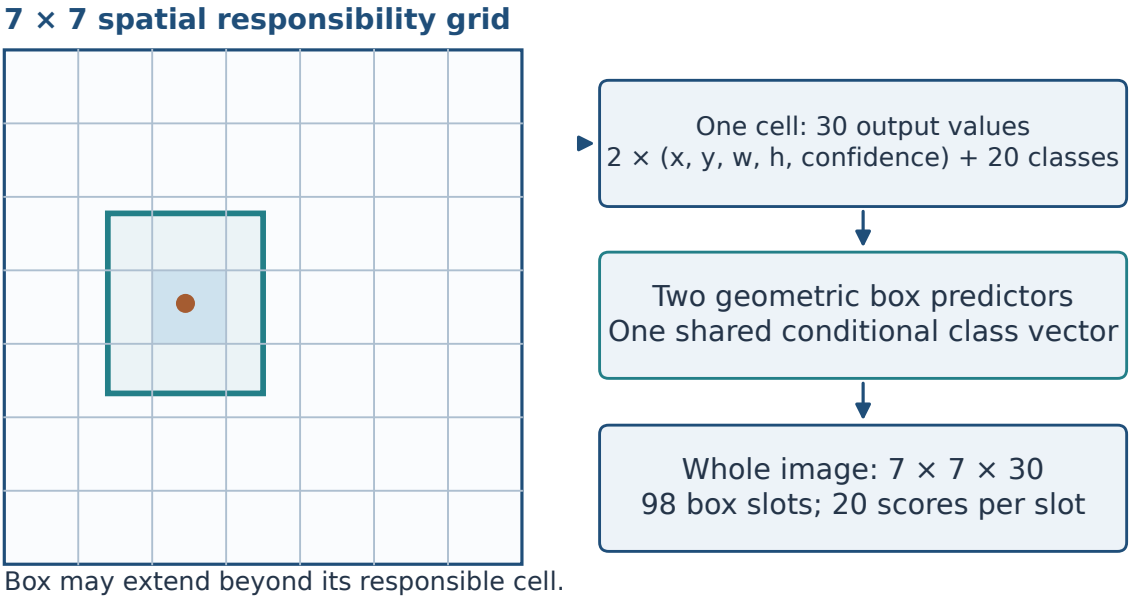


Figure 2: Grid responsibility and the $7 \times 7 \times 30$ representation. The marked center assigns the illustrated box to one cell; the box itself spans several cells. Original schematic based on [1].

For example, let an object's center lie at image-normalized coordinates $(0.35, 0.493)$. With zero-based indexing, it belongs to column $\lfloor 7(0.35) \rfloor = 2$ and row $\lfloor 7(0.493) \rfloor = 3$. The corresponding local offsets are approximately $(0.45, 0.45)$. A width of 0.30 describes 30% of the image width, not 30% of the cell width. This distinction is essential for correct decoding: grid-relative center offsets and image-relative sizes use different reference frames.

3.3 Network architecture

YOLOv1 uses 24 convolutional layers followed by two fully connected layers [1]. Its convolutional backbone is inspired by GoogLeNet, but it does not use Inception modules. Instead, it often uses a 1×1 convolutional reduction layer before a more expensive 3×3 convolution. This reduces channel dimensionality and computational cost while retaining learned combinations of input features.

At a high level, an input image of $448 \times 448 \times 3$ is transformed into a $7 \times 7 \times 1024$ feature representation. The fully connected layers map this representation through a 4,096-unit hidden layer to the final $7 \times 7 \times 30$ detection tensor. The progressive reduction in spatial resolution provides semantic abstraction, but it also contributes to the model's coarse localization.

The architecture separates feature extraction from global prediction. The 1×1 layers mix channels and reduce intermediate width, while 3×3 layers extract spatial patterns. Four pooling

Table 1: High-level YOLOv1 architecture for the PASCAL VOC configuration.

Stage	Main role	Output
Input	RGB image	$448 \times 448 \times 3$
Convolutional backbone	Feature extraction (24 convolutional layers)	Approx. $7 \times 7 \times 1024$
Fully connected layer	Global hidden representation	4096
Prediction layer	Boxes, confidences, and class probabilities	$7 \times 7 \times 30$



Figure 3: Reserved space for the YOLOv1 architecture diagram.

stages and a later stride-2 convolution progressively reduce resolution. Flattening the final feature representation enables fully connected prediction using global information, but ties the predictor to a fixed feature size. Thus, resizing an arbitrary image to 448×448 is part of this original configuration; it is not a fully convolutional, arbitrary-resolution detector.

Fast YOLO is a separate, smaller network with nine convolutional layers and two fully connected layers. Its higher speed is obtained through reduced architectural capacity, with a corresponding loss of accuracy [1]. It should not be described as merely a faster runtime setting of the 24-layer model.

3.4 Training strategy

3.4.1 Pretraining and detection conversion

The first 20 convolutional layers are pretrained on the ImageNet 1000-class classification task using 224×224 inputs. The detector is then formed by adding four convolutional layers and two randomly initialized fully connected layers. Training for detection uses 448×448 inputs, which supplies more spatial detail for localization [1].

Hidden layers use the leaky rectified linear unit (Leaky ReLU),

$$\phi(z) = \begin{cases} z, & z > 0, \\ 0.1z, & z \leq 0, \end{cases} \tag{8}$$

while the final layer is linear. The linear output layer allows continuous regression to coordinates and confidence-related values; the desired ranges are encouraged by target encoding and loss optimization rather than enforced by a final bounding activation.

3.4.2 Loss function and responsibility assignment

YOLOv1 uses a re-weighted, conditional sum-squared-error (SSE) objective. Its logic can be summarized as

$$\mathcal{L} = 5\mathcal{L}_{xy} + 5\mathcal{L}_{\sqrt{w},\sqrt{h}} + \mathcal{L}_{\text{conf,obj}} + 0.5\mathcal{L}_{\text{conf,noobj}} + \mathcal{L}_{\text{class}}. \tag{9}$$

The coordinate weight $\lambda_{\text{coord}} = 5$ increases the influence of localization error. The no-object confidence weight $\lambda_{\text{noobj}} = 0.5$ prevents the many background predictions from overwhelming object-related gradients. Square roots of w and h make the objective relatively more sensitive to size errors for small boxes.

For an object-bearing cell, only the box predictor with the highest current IoU against the ground-truth box becomes responsible for coordinate and object-confidence learning. This assignment encourages the two predictors to specialize in different box shapes, sizes, or patterns. Classification loss is active only when an object is present in the cell.

For clarity, let i index cells, j index box predictors, and k index classes. Predictions have no hat and targets have a hat. Let $\mathbb{1}_i^{\text{obj}}$ denote an object-bearing cell, $\mathbb{1}_{ij}^{\text{obj}}$ its responsible predictor, and $\mathbb{1}_{ij}^{\text{noobj}}$ a predictor assigned a no-object confidence target. The full objective is

$$\begin{aligned}
\mathcal{L} = {} & \lambda_{\text{coord}} \sum_{i,j} \mathbb{1}_{ij}^{\text{obj}} [(x_{ij} - \hat{x}_i)^2 + (y_{ij} - \hat{y}_i)^2] \\
& + \lambda_{\text{coord}} \sum_{i,j} \mathbb{1}_{ij}^{\text{obj}} [(\sqrt{w_{ij}} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_{ij}} - \sqrt{\hat{h}_i})^2] \\
& + \sum_{i,j} \mathbb{1}_{ij}^{\text{obj}} (C_{ij} - \hat{C}_{ij})^2 + \lambda_{\text{noobj}} \sum_{i,j} \mathbb{1}_{ij}^{\text{noobj}} C_{ij}^2 \\
& + \sum_{i} \mathbb{1}_{i}^{\text{obj}} \sum_{k=1}^{20} (p_i(k) - \hat{p}_i(k))^2.
\end{aligned} \tag{10}$$

Here, $\hat{C}_{ij}$ is the IoU target for the responsible predictor, while a no-object confidence target is zero. The notation describes positive box sizes in the paper's loss; implementations that output square-root size parameters must decode those parameters consistently. It does not imply a sigmoid-constrained final layer. The indicator structure in equation (10) prevents background cells from contributing coordinate or class loss.

3.4.3 Optimization and regularization

The authors train for approximately 135 epochs on PASCAL VOC 2007 and 2012 training/validation data. The reported batch size is 64, momentum is 0.9, and weight decay is 0.0005. The learning rate is warmed up from 10^{-3} to 10^{-2}, then maintained at 10^{-2} for 75 epochs, reduced to 10^{-3} for 30 epochs, and finally reduced to 10^{-4} for 30 epochs [1].

To reduce overfitting, YOLOv1 applies dropout with rate 0.5 after the first fully connected layer. Data augmentation includes random scaling and translation up to 20% of image size, together with random exposure and saturation changes in HSV space.

3.5 Inference and post-processing

At test time, one forward pass generates $7 \times 7 \times 2 = 98$ candidate boxes. Each candidate is combined with the class-probability vector from its cell to obtain class-specific scores. Low-scoring predictions are discarded, and non-maximum suppression (NMS) removes duplicate overlapping boxes of the same class. The score threshold and NMS IoU threshold are distinct hyperparameters.

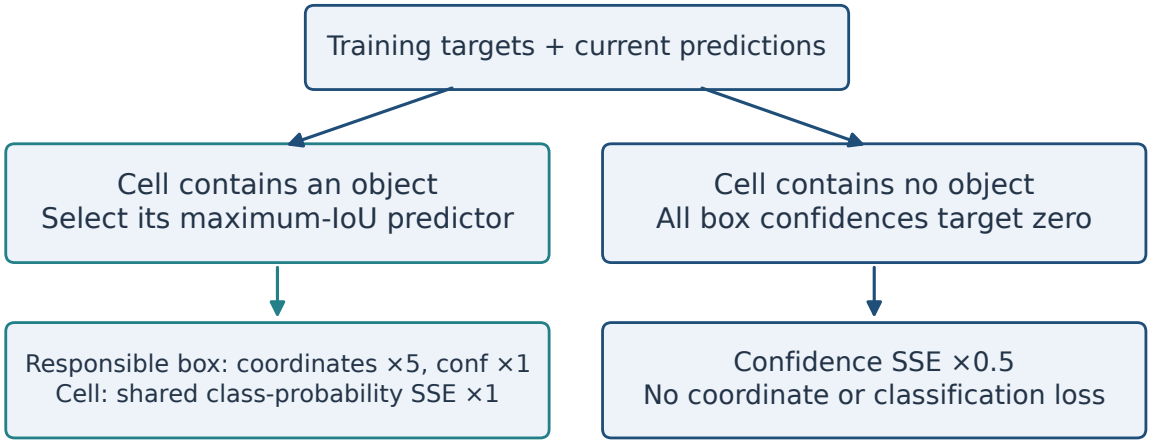


Figure 4: Conditional learning signals in YOLOv1: object presence activates the class term, while maximum-IoU responsibility selects the coordinate-learning predictor. Original schematic based on the loss in [1].

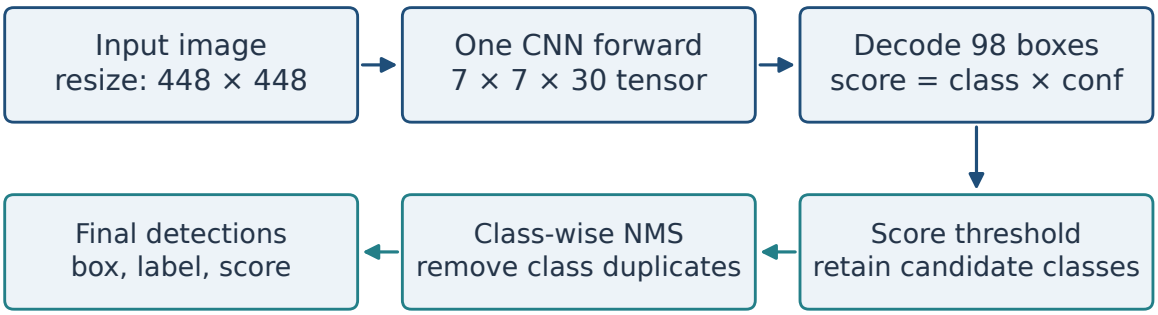


Figure 5: YOLOv1 inference, including box decoding and post-processing. The forward network is evaluated once, but score filtering and NMS remain separate operations. Original schematic based on [1].

Ground-truth boxes are not available during inference. The network predicts the confidence value that was trained toward $P(\text{Object}) \times \text{IoU}$; it does not calculate an IoU against ground truth at test time. This distinction avoids incorrectly introducing labels into the inference pipeline. NMS instead compares pairs of predicted boxes, retaining high-scoring predictions and suppressing lower-scoring overlaps within the same class.

4 Key Innovations Relative to Earlier Detectors

YOLOv1's central innovation is a system-level formulation rather than a new convolution operator. Its explicit grid representation makes detection a fixed-size prediction problem, and its shared network learns localization and classification together. Table 2 contrasts the computation patterns of representative preceding detectors. The comparison concerns their original designs, not all modern two-stage systems.

Table 2: Prediction workflows before and in YOLOv1 [2, 3, 1].

Design	Candidate generation	Feature computation	Prediction unit
R-CNN	External region proposals	Separate CNN evaluation for crops	Proposed region
Fast R-CNN	External region proposals	Shared feature map; RoI pooling	Proposed region
YOLOv1	No external proposals	One full-image CNN evaluation	Grid cell and its two box predictors

The resulting advantages are connected: avoiding per-crop computation reduces repeated work; global features supply context; and one detection objective coordinates the learned components. However, a fixed output representation transfers complexity into the model's capacity constraints. In particular, two predictors in one cell provide alternative geometry, not two fully independent class distributions. The formulation is therefore efficient without being unrestricted.

5 Experimental Results and Assessment

5.1 Reported performance

The original paper reports a clear speed–accuracy trade-off on PASCAL VOC 2007. Standard YOLO reaches 63.4% mean average precision (mAP) at 45 frames per second (FPS), whereas Fast YOLO reaches 52.7% mAP at 155 FPS [1]. These historical values should be interpreted in the hardware and benchmark context used by the original paper, rather than as direct comparisons to current detectors.

Table 3: Reported YOLOv1 results on PASCAL VOC 2007 [1].

Model	mAP (%)	FPS
Fast YOLO	52.7	155
YOLO	63.4	45

Mean average precision summarizes class-wise precision–recall performance. In the VOC 2007 evaluation used here, a detection must match a ground-truth object at an IoU threshold of 0.5; VOC 2007 uses its historical 11-point interpolated AP calculation [4]. These values should not be mixed with COCO AP averaged over thresholds from 0.50 to 0.95. FPS is throughput in the original experimental setting, not a guaranteed latency on another machine.

The paper reports GPU timing on an NVIDIA Titan X and evaluates different detectors with

different architectures and training histories. Accordingly, table 3 records published evidence rather than a reproduction. A fair new comparison would hold dataset split, resolution, hardware, batch size, numerical precision, runtime, and measured pipeline stages constant. No new VOC accuracy or neural-network timing is claimed in this report.

5.2 Advantages

The original model offers several advantages supported by the formulation and the paper's evaluation:

- **Unified prediction:** bounding boxes and class probabilities are produced by a single network evaluation.
- **Real-time throughput:** the detector avoids processing thousands of proposal regions independently.
- **Global context:** predictions use information from the complete image, reducing background false positives relative to Fast R-CNN in the original analysis.
- **Joint optimization:** the core detector can be optimized end-to-end for the detection task.
- **Generalization evidence:** the original paper found that YOLO degraded less than R-CNN on artwork datasets, suggesting useful global object representations.

5.3 Limitations

The same design choices that make YOLOv1 fast also impose important constraints:

- Each cell predicts only two boxes and one shared class distribution, limiting the number of nearby objects it can represent.
- Several small or clustered objects with centers in the same cell are particularly difficult to detect.
- Repeated downsampling yields relatively coarse features for precise localization.
- The model can struggle with unusual aspect ratios, poses, and object configurations not well represented during training.
- The re-weighted SSE objective is a practical surrogate, but it is not perfectly aligned with AP or mAP, the final evaluation metrics.

The paper's error analysis identifies localization as YOLOv1's dominant source of error. These shortcomings provide useful context for understanding why later YOLO variants modify prediction heads, feature extraction, assignment strategies, and loss functions.

The limitations also explain why adding a second box predictor does not solve crowded-scene detection. If two nearby objects fall in one cell, their class supervision competes for one shared vector and the simple target assignment cannot represent an arbitrary set of instances. Increasing input detail alone cannot remove this output-capacity restriction. Meanwhile, the square-root size loss reduces one imbalance but still differs from directly optimizing box overlap: a small coordinate error can cause a large IoU decrease for a small object.

A geometric example makes this scale sensitivity concrete. Consider a square ground-truth box of side length a and an equally sized prediction shifted horizontally by $d < a$. Their IoU is $(a - d)/(a + d)$. A shift of two pixels gives IoU $8/12 \approx 0.667$ for a ten-pixel square, but $98/102 \approx 0.961$ for a hundred-pixel square. Identical absolute coordinate errors can therefore have very different evaluation consequences. This example is a geometric calculation, not a measured failure from the network.

The confidence definition also creates a distinction between object presence and box quality. A low score can reflect poor expected overlap as well as weak evidence for the class. Increasing the score threshold may hide imprecise boxes, but can remove true detections rather than correcting

localization. Conversely, a permissive threshold followed by NMS can reduce duplicates without repairing their coordinates. These mechanisms make qualitative inspection of boxes important alongside aggregate mAP; score filtering alone cannot overcome the representation and loss limitations described above.

6 Typical Applications of YOLOv1

The paper motivates fast object detection for responsive systems such as autonomous driving, assistive vision, robotics, and real-time webcam or video analysis [1]. A deployment decision should nevertheless account for the detector's known weaknesses in small-object and crowded-scene localization.

For a webcam demonstration or robot perception prototype, the appeal is a single forward pass that can update detections frequently. Video analysis can use the boxes as inputs to a separate tracking system, but YOLOv1 itself does not maintain identities across frames. Assistive perception benefits from low delay, yet distance estimation or action selection requires additional components beyond 2D boxes. Vehicle-scene analysis is another motivated application, although distant pedestrians and clustered road users directly expose the model's weaknesses.

These examples describe potential uses of the original detector, not claims that the paper validated every deployment. The VOC configuration covers 20 known categories and predicts axis-aligned boxes; it does not intrinsically provide instance masks, pose keypoints, unknown-category recognition, or temporal tracking. The suitability of an application therefore depends on its target classes, object scales, scene density, and acceptable localization error.

7 Simple Demonstration / Implementation Example

Reserved for later completion. The implementation has not yet been performed.

Real-image inference example to be added.

Figure 6: Reserved space for input images and YOLOv1 inference results.

8 Evolution Beyond YOLOv1

8.1 Reading the family as several research branches

Later versions provide context for YOLOv1 rather than a single organizational lineage. Redmon and Farhadi developed v2/v3; later models came from several research teams. Table 4 records their sources. The progression in figure 7 summarizes technical themes rather than strict architectural inheritance.

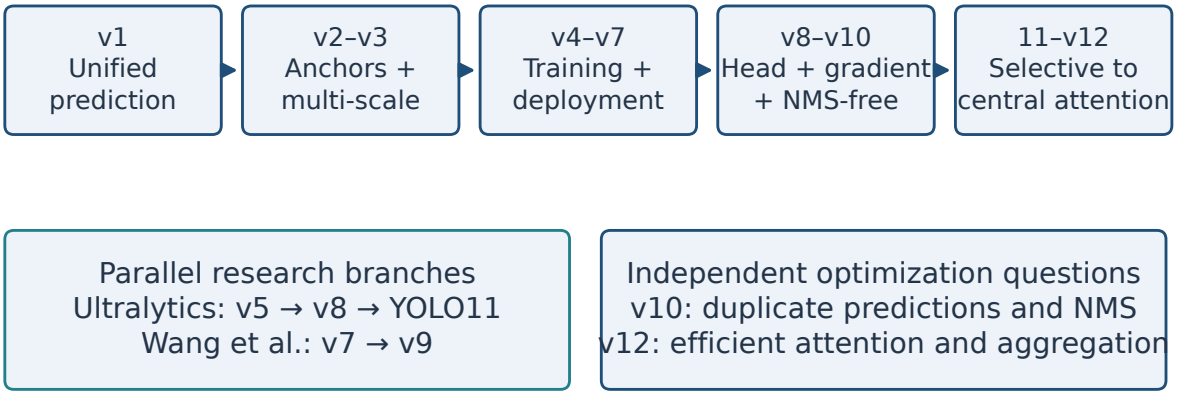


Figure 7: A concise map of technical themes beyond YOLOv1. Parallel branches pursue different combinations of architecture, training, and deployment improvements; the diagram summarizes the sources cited in this section.

8.2 From box priors to multi-scale representation: v2–v3

YOLOv2 improves recall and localization with anchor boxes, dimension clustering, constrained center prediction, batch normalization, and Darknet-19. Passthrough features and multi-scale training improve flexibility. YOLO9000 additionally combines classification and detection supervision through hierarchical labels [5].

YOLOv3 combines residual Darknet-53 with three prediction scales. Upsampling and concatenation preserve semantic and spatial information, helping small-object detection. Independent logistic predictions allow overlapping class labels. Weaker performance at stricter IoU thresholds shows that precise localization remains challenging [6]. Together, v2/v3 address YOLOv1's coarse representation.

8.3 Optimizing training and deployment: v4–v7

YOLOv4 combines CSPDarknet53, spatial pyramid pooling, and path aggregation. Its distinction between bag-of-freebies and bag-of-specials separates training improvements from modifications that also affect inference. Mosaic augmentation and CIoU-based localization illustrate how better supervision can complement better features [7].

YOLOv5 develops a practical PyTorch pipeline with scaled model sizes. Later v5 revisions use C3 blocks and SPPF; AutoAnchor and training refinements support adaptation to a dataset. These revision-specific components should not all be attributed to the initial 2020 release [8, 9]. Here, v5 means the original anchor-based family, not the later anchor-free v5u variant.

YOLOv6 targets industrial efficiency using re-parameterizable feature extraction and an anchor-free decoupled head. Its v3.0 update adds bidirectional feature fusion, anchor-aided training, and localization distillation, keeping auxiliary training machinery out of the inference path [10, 11]. YOLOv7 instead emphasizes layer aggregation, architecture-aware scaling, planned re-parameterization, and lead-head-guided auxiliary supervision [12]. These parallel designs show that richer training need not require a correspondingly larger deployed graph.

8.4 Revisiting the head, gradient, and post-processing: v8–v10

YOLOv8's Ultralytics branch uses C2f blocks and an anchor-free split head. Task-aligned assignment and distribution-based box regression accompany its detection implementation [13, 14]. Removing predefined box shapes simplifies one modeling choice; it does not eliminate multiple predictions for one object. Its normal detection path still applies NMS.

YOLOv9 addresses unreliable learning signals caused by information loss through deep transformations. Programmable Gradient Information (PGI) supplies auxiliary supervision, while GELAN organizes efficient feature aggregation. The auxiliary structure supports training and can be removed for inference [15]. This extends concerns about gradient paths rather than merely adding more prediction layers.

YOLOv10 targets the cost of duplicate predictions and NMS. Consistent dual assignments combine one-to-many supervision with a one-to-one head; inference retains the latter and does not need NMS. Lightweight components and selective larger-context operations further optimize the accuracy–latency trade-off [16]. In contrast to YOLOv1's jointly learned prediction network, this use of end-to-end detection also removes NMS from the output pipeline.

8.5 Toward attention-centric features: YOLO11–v12

YOLO11 refines the Ultralytics line with C3k2 and C2PSA modules, adding selective attention while retaining a conventional anchor-free detection head [17, 14]. It does not automatically inherit YOLOv10's NMS-free assignment. Official software documentation supplies the references for YOLOv8 and YOLO11, since Ultralytics has not released formal original research papers for these models.

YOLOv12 makes efficient attention a central design concern through Area Attention and residual efficient layer aggregation (R-ELAN). Partitioning features into large areas reduces attention cost; residual aggregation supports stable optimization, and FlashAttention reduces memory-access overhead [18]. With a fixed number of areas, this reduces the coefficient of quadratic interaction cost rather than making attention linear in token count. Its benefits remain tied to implementation and hardware conditions.

The optimization scope expands from YOLOv1's output formulation to features, supervision, deployment, and context. Comparisons still require compatible AP definitions and latency conditions; the appendix catalogues contributions without ranking heterogeneous benchmarks.

9 Conclusion

YOLOv1 demonstrates that object detection can be formulated as a single, global prediction problem. Its grid-based output, shared full-image features, and jointly trained detector provide a clear route to real-time inference. The same design restricts nearby-object capacity and precise localization, making the original paper useful both as a conceptual foundation and as an explanation of subsequent research needs. Later YOLO branches show how the common speed–accuracy objective can motivate changes to representation, supervision, post-processing, and attention rather than one universal sequence of architectural replacements. Implementation and qualitative inference results remain pending and will be added when supporting materials are available.

A YOLO Family Catalogue: v1–v12

The dates distinguish initial preprint/release from conference publication where relevant. The final column describes the detection problem or deployment emphasis; it is not a claim that each paper validated every downstream application. YOLO11 is the official Ultralytics name. All models listed concern fast object detection; some software families also provide separate variants for other tasks.

Table 4: Release history, responsible groups, critical improvements, and problem emphasis across the YOLO family.

Version / source	First appearance	Individual / organization	Main characteristic or critical improvement	Target problem / task emphasis
YOLOv1 [1]	2015 preprint; 2016 CVPR	Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi; University of Washington / AI2 / Facebook AI Research	Single full-image prediction; 7×7 grid; 24 conv + 2 FC; shared per-cell class vector	Replace proposal-based prediction with real-time, unified bounding-box detection.
YOLOv2 [5]	2016 preprint; 2017 CVPR	Joseph Redmon and Ali Farhadi; University of Washington / AI2 / XNOR.ai	Darknet-19, learned anchor priors, stable center prediction, BN, multi-scale training; YOLO9000 joint supervision	Improve recall and localization; YOLO9000 additionally broadens category coverage using classification data.
YOLOv3 [6]	2018	Joseph Redmon and Ali Farhadi; University of Washington	Residual Darknet-53; three prediction scales; independent logistic class outputs	Improve scale-dependent detection, particularly small objects, while retaining speed.
YOLOv4 [7]	2020	Alexey Bochkovskiy, Chien-Yao Wang, Hong-Yuan Mark Liao; research collaboration including Academia Sinica	CSPDarknet53, SPP, PAN; coordinated BoF/BoS, Mosaic and CIoU	Improve the accuracy–speed balance with feasible single-GPU training.
YOLOv5 [8, 9]	2020 repository	Ultralytics; led by Glenn Jocher	PyTorch pipeline; scaled sizes; original anchor-based head; later revisions use C3/SPPF and AutoAnchor	Make custom-dataset training, validation, export, and deployment practical.

Continued on the next page

Table 4 continued

Version / source	First appearance	Individual / organization	Main characteristic or critical improvement	Target problem / task emphasis
YOLOv6 [10, 11]	2022; v3.0 in 2023	Chuyi Li and collaborators; Meituan	EfficientRep, re-parameterization, decoupled anchor-free head; v3.0 adds RepBi-PAN, AAT, DLD	Hardware-efficient industrial detection and deployment; retain richer supervision without auxiliary inference cost.
YOLOv7 [12]	2022 preprint; 2023 CVPR	Chien-Yao Wang, Alexey Bochkovskiy, Hong-Yuan Mark Liao; Academia Sinica collaboration	E-ELAN, concatenation-aware scaling, planned re-parameterization, trainable bag-of-freebies	Improve feature learning and speed–accuracy scaling through architecture and auxiliary training.
YOLOv8 [13, 14]	January 2023	Ultralytics; Glenn Jocher, Ayush Chaurasia, Jing Qiu and contributors	C2f, anchor-free split head, distributional regression; unified API	Simplify detection training/deployment; separate variants support segmentation, pose, classification, and OBB.
YOLOv9 [15]	2024	Chien-Yao Wang, I-Hau Yeh, Hong-Yuan Mark Liao; Academia Sinica / National Taipei University of Technology	Programmable Gradient Information and GELAN	Address information loss and improve reliable learning signals in efficient detection networks.
YOLOv10 [16]	2024	Ao Wang and collaborators; Tsinghua University-led research team	Consistent dual assignments, one-to-one inference, NMS-free output; efficiency-driven component design	Reduce duplicate prediction and post-processing latency for end-to-end real-time detection.
YOLO11 [17, 14]	September 2024	Ultralytics; Glenn Jocher, Jing Qiu and contributors	C3k2, C2PSA, refined feature extraction; conventional anchor-free detection head	Improve efficient detection in the Ultralytics ecosystem; distinct variants cover multiple vision tasks.
YOLOv12 [18]	February 2025	Yunjie Tian, Qixiang Ye, David Doermann; University at Buffalo / University of Chinese Academy of Sciences	Area Attention, R-ELAN, attention-oriented optimization, FlashAttention	Capture broader feature interactions while constraining latency and stabilizing attention-based training.

References

[1] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.

[2] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 580–587, 2014.

[3] Ross Girshick. Fast R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1440–1448, 2015.

[4] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The PASCAL visual object classes (VOC) challenge. *International Journal of Computer Vision*, 88:303–338, 2010.

[5] Joseph Redmon and Ali Farhadi. YOLO9000: Better, faster, stronger. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7263–7271, 2017.

[6] Joseph Redmon and Ali Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[7] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[8] Ultralytics. YOLOv5: Repository and v1.0 release notes. https://github.com/ultralytics/yolov5/releases/tag/v1.0, 2020. Accessed October 13, 2026.

[9] Ultralytics. YOLOv5 architecture description. https://docs.ultralytics.com/yolov5/tutorials/architecture-description/. Describes the v6.0/v6.1 implementation; accessed October 13, 2026.

[10] Chuyi Li, Lulu Li, Hongliang Jiang, et al. YOLOv6: A single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022.

[11] Chuyi Li, Lulu Li, Yifei Geng, et al. YOLOv6 v3.0: A full-scale reloading. *arXiv preprint arXiv:2301.05586*, 2023.

[12] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7464–7475, 2023.

[13] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8: Model documentation. https://docs.ultralytics.com/models/yolov8/, 2023. Official software documentation; accessed October 13, 2026.

[14] Ultralytics. Ultralytics model configurations and detection modules, tag v8.3.0. https://github.com/ultralytics/ultralytics/tree/v8.3.0/ultralytics, 2024. YOLOv8/YOLO11 configuration, head, block, loss, and NMS modules; accessed October 13, 2026.

[15] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. YOLOv9: Learning what you want to learn using programmable gradient information. *arXiv preprint arXiv:2402.13616*, 2024.

[16] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. YOLOv10: Real-time end-to-end object detection. *Advances in Neural Information Processing Systems*, 37, 2024. Also available as arXiv:2405.14458.

[17] Glenn Jocher and Jing Qiu. Ultralytics YOLO11: Model documentation. https://docs.ultralytics.com/models/yolo11/, 2024. Official software documentation; accessed October 13, 2026.

[18] Yunjie Tian, Qixiang Ye, and David Doermann. YOLOv12: Attention-centric real-time object detectors. *arXiv preprint arXiv:2502.12524v1*, 2025. Technical report, February 18, 2025.